US 20250286707A1

(19) United States

(12) Patent Application Publication (10) Pub. No.: US 2025/0286707 A1

LEUNER et al. (43) Pub. Date: Sep. 11, 2025

(54) **MULTIMODAL CRYPTOGRAPHIC SYSTEM, COMPUTER EXECUTABLE INSTRUCTIONS AND METHOD**

(71) Applicant: **evolutionQ Inc.**, Waterloo (CA)

(72) Inventors: **Martin LEUNER**, Aachen (DE); **Patrick REINELT**, Aachen (DE); **Michele MOSCA**, Kitchener (CA); **Thorsten Heiner GRÖTKER**, Aachen (DE); **David Yen JAO**, Waterloo (CA)

(73) Assignee: **evolutionQ Inc.**, Waterloo (CA)

(21) Appl. No.: **19/215,826**

(22) Filed: **May 22, 2025**

Related U.S. Application Data

(63) Continuation of application No. PCT/CA2024/050190, filed on Feb. 15, 2024.

(60) Provisional application No. 63/485,050, filed on Feb. 15, 2023, provisional application No. 63/519,945, filed on Aug. 16, 2023.

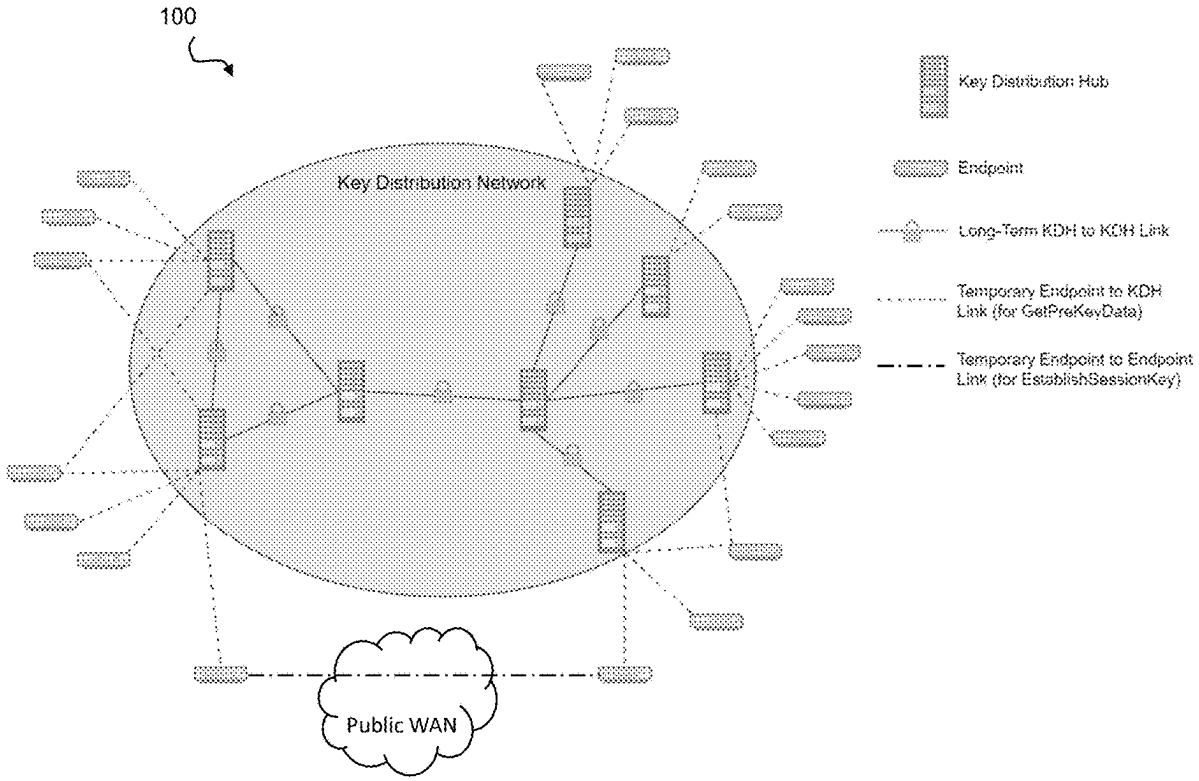
Publication Classification

(51) **Int. Cl.**
H04L 9/08 (2006.01)

(52) **U.S. Cl.**
CPC ***H04L 9/085*** (2013.01); ***H04L 9/0819*** (2013.01); ***H04L 9/0852*** (2013.01)

(57) **ABSTRACT**

A method, system and computer readable medium for establishment of cryptographic secrets is disclosed. Illustratively, the method includes obtaining a first input share based on a hybrid key establishment method, obtaining a second input share from a key distribution network, and deriving, from the first input share and the second input share, a shared secret for use in cryptographic communication between an initiator and a respondent.



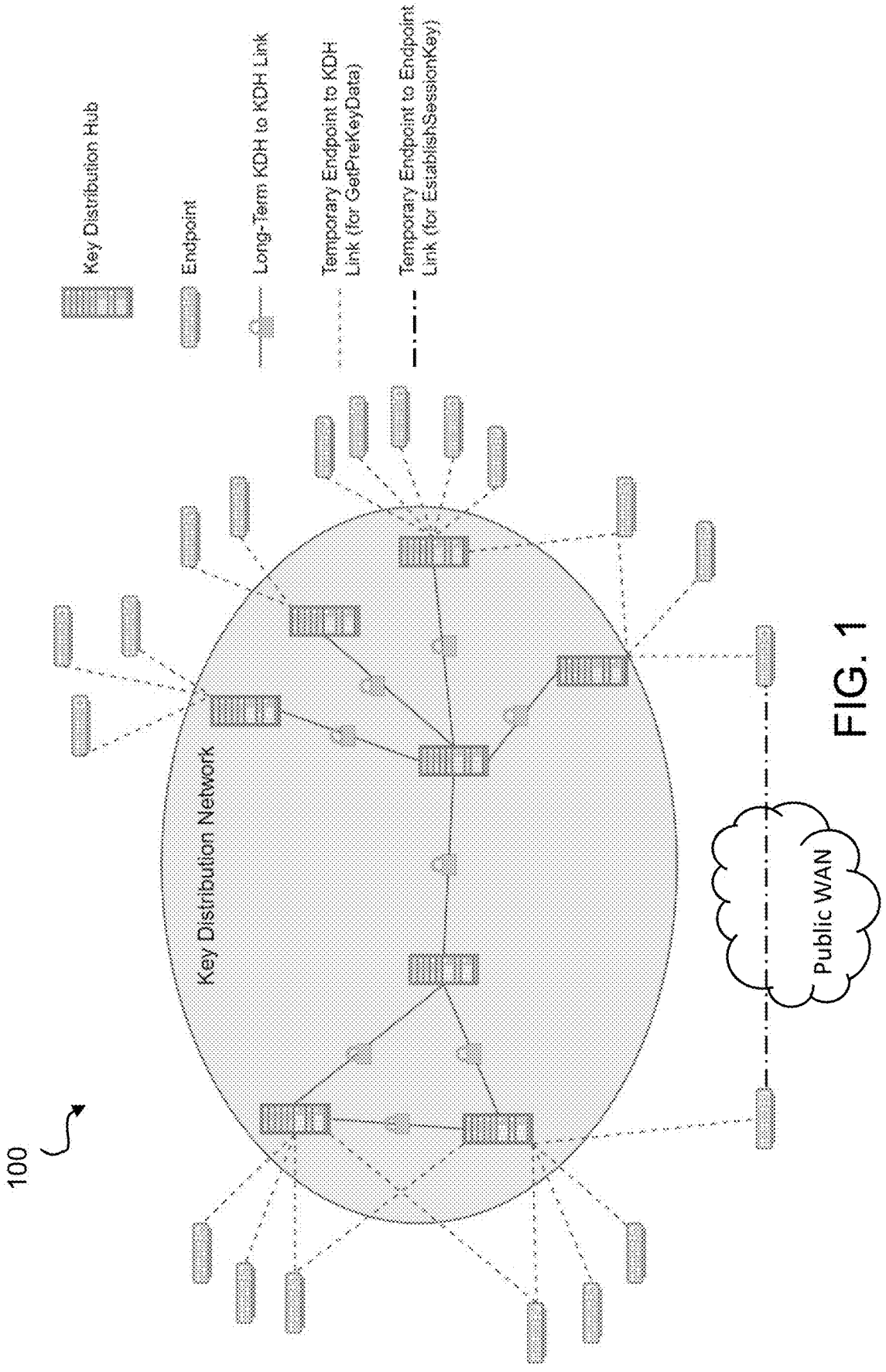

Key Distribution Hub
Endpoint
Long-Term KDH to KDH Link
Temporary Endpoint to KDH Link (for GetPreKeyData)
Temporary Endpoint to Endpoint Link (for EstablishSessionKey)
Key Distribution Network
Public WAN
100

FIG. 1

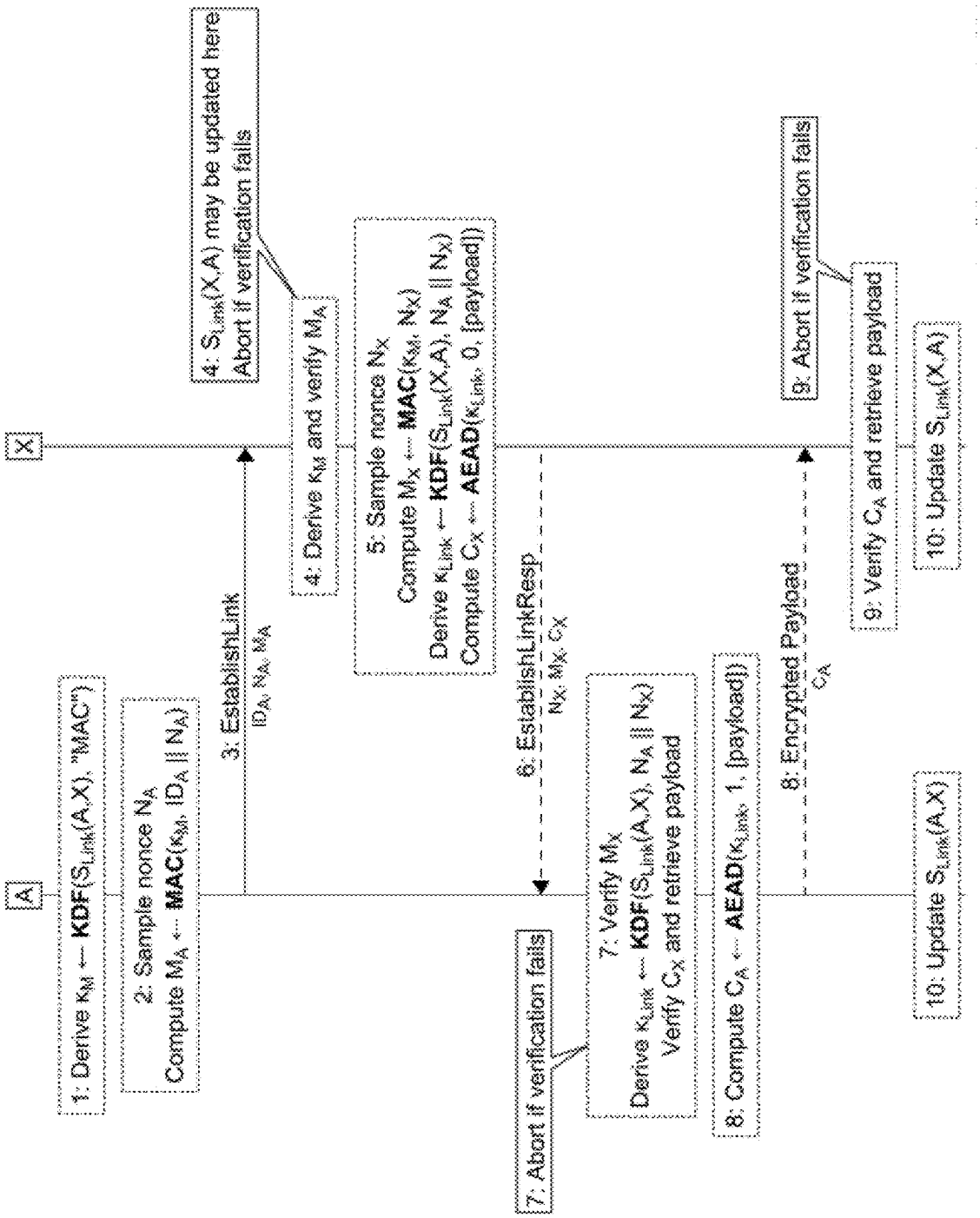

A
X
1: Derive $\kappa_M \leftarrow$ **KDF**(S_{Link}(A,X), "MAC")
2: Sample nonce N_A
Compute $M_A \leftarrow$ **MAC**(κ_M, ID_A || N_A)
3: EstablishLink
ID_A, N_A, M_A
4: S_{Link}(X,A) may be updated here
Abort if verification fails
4: Derive κ_M and verify M_A
5: Sample nonce N_X
Compute $M_X \leftarrow$ **MAC**(κ_M, N_X)
Derive $\kappa_{Link} \leftarrow$ **KDF**(S_{Link}(X,A), N_A || N_X)
Compute $C_X \leftarrow$ **AEAD**(κ_{Link}, 0, {payload})
6: EstablishLinkResp
N_X, M_X, C_X
7: Abort if verification fails
7: Verify M_X
Derive $\kappa_{Link} \leftarrow$ **KDF**(S_{Link}(A,X), N_A || N_X)
Verify C_X and retrieve payload
8: Compute $C_A \leftarrow$ **AEAD**(κ_{Link}, 1, {payload})
8: Encrypted Payload
C_A
9: Abort if verification fails
9: Verify C_A and retrieve payload
10: Update S_{Link}(X,A)
10: Update S_{Link}(A,X)

FIG. 2

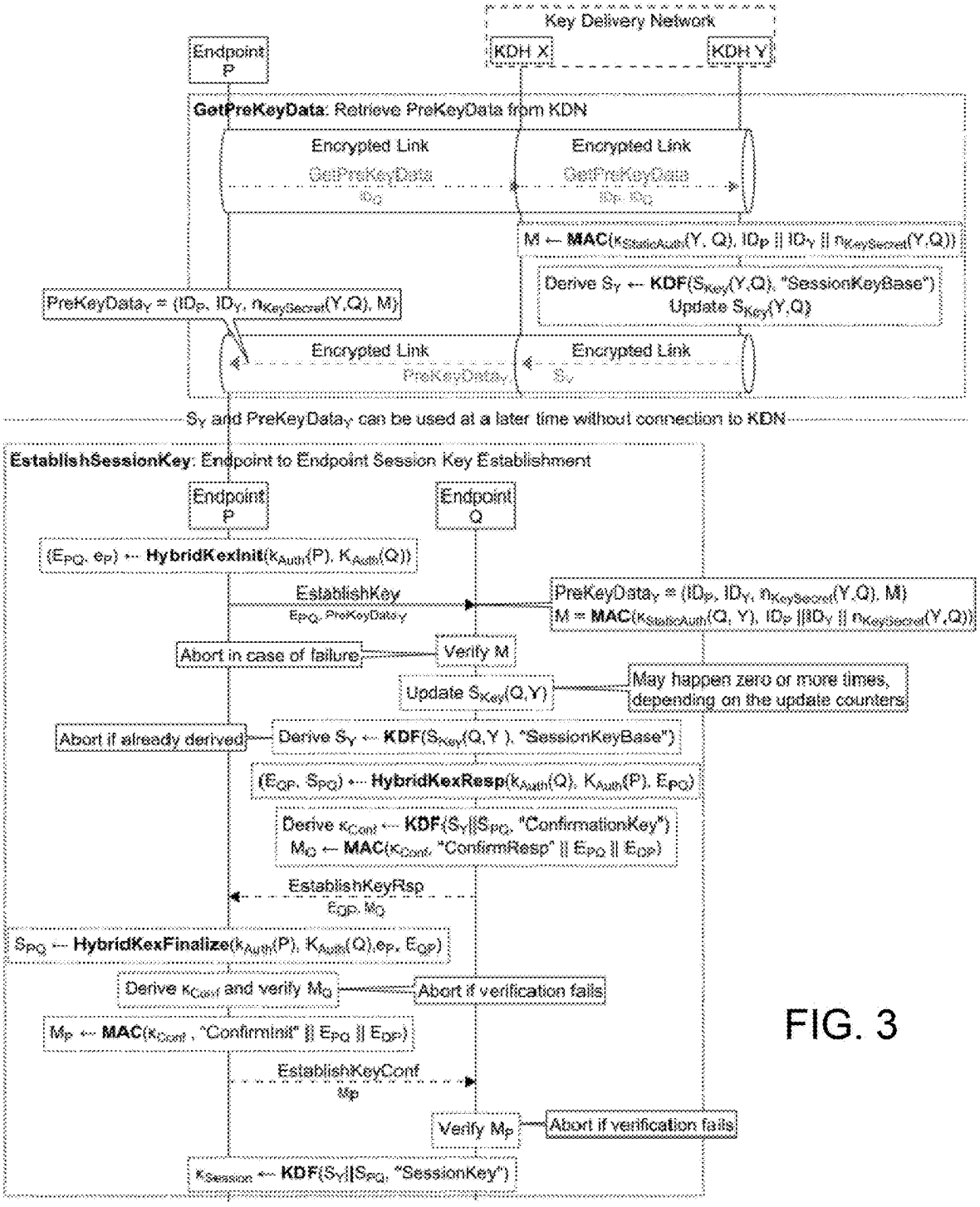

Key Delivery Network
KDH X
KDH Y
Endpoint P
GetPreKeyData: Retrieve PreKeyData from KDN
Encrypted Link
GetPreKeyData
Encrypted Link
GetPreKeyData
M ← MAC(K_StaticAuth(Y, Q), ID_P || ID_Y || n_KeySecret(Y,Q))
Derive S_Y ← KDF(S_Key(Y,Q), "SessionKeyBase")
Update S_Key(Y,Q)
PreKeyData_Y = (ID_P, ID_Y, n_KeySecret(Y,Q), M)
Encrypted Link
PreKeyData_Y
Encrypted Link
S_Y
S_Y and PreKeyData_Y can be used at a later time without connection to KDN
EstablishSessionKey: Endpoint to Endpoint Session Key Establishment
Endpoint P
Endpoint Q
(E_PQ, e_P) ← HybridKexInit(K_Auth(P), K_Auth(Q))
EstablishKey
E_PQ, PreKeyData_Y
PreKeyData_Y = (ID_P, ID_Y, n_KeySecret(Y,Q), M)
M = MAC(K_StaticAuth(Q, Y), ID_P ||ID_Y || n_KeySecret(Y,Q))
Abort in case of failure
Verify M
Update S_Key(Q,Y)
May happen zero or more times, depending on the update counters
Abort if already derived
Derive S_Y ← KDF(S_Key(Q,Y), "SessionKeyBase")
(E_QP, S_PQ) ← HybridKexResp(K_Auth(Q), K_Auth(P), E_PQ)
Derive K_Conf ← KDF(S_Y||S_PQ, "ConfirmationKey")
M_Q ← MAC(K_Conf, "ConfirmResp" || E_PQ || E_QP)
EstablishKeyRsp
E_QP, M_Q
S_PQ ← HybridKexFinalize(K_Auth(P), K_Auth(Q),e_P, E_QP)
Derive K_Conf and verify M_Q
Abort if verification fails
M_P ← MAC(K_Conf , "ConfirmInit" || E_PQ || E_DP)
EstablishKeyConf
M_P
Verify M_P
Abort if verification fails
K_Session ← KDF(S_Y|S_PQ, "SessionKey")

FIG. 3

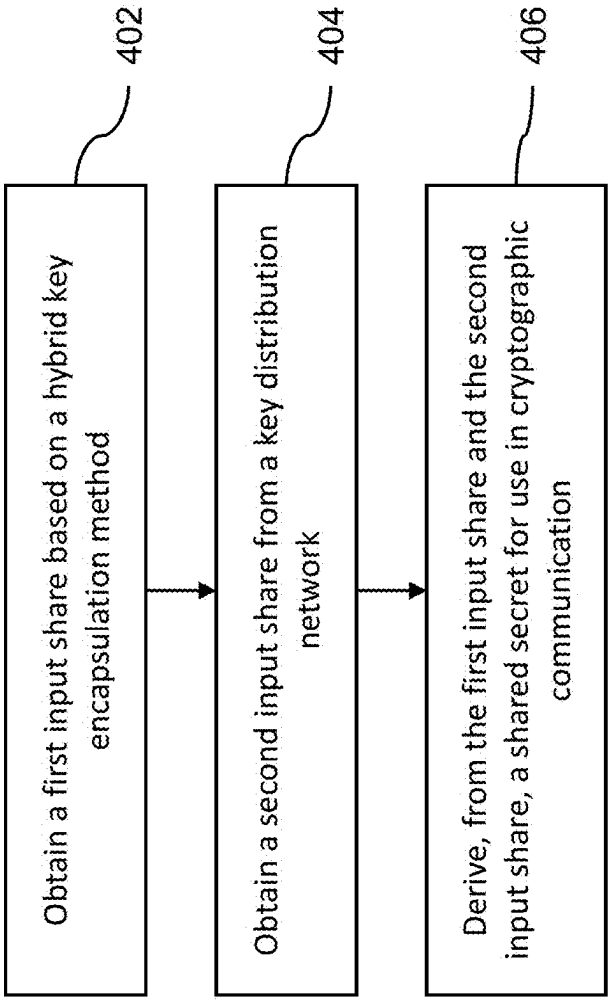

402
Obtain a first input share based on a hybrid key encapsulation method
404
Obtain a second input share from a key distribution network
406
Derive, from the first input share and the second input share, a shared secret for use in cryptographic communication

FIG. 4

MULTIMODAL CRYPTOGRAPHIC SYSTEM, COMPUTER EXECUTABLE INSTRUCTIONS AND METHOD

CROSS REFERENCE TO RELATED APPLICATIONS

[0001] This application is a Continuation of PCT Patent Application No. PCT/CA2024/050190, filed on Feb. 15, 2024, which claims priority to U.S. Provisional Patent Application No. 63/485,050 filed on Feb. 15, 2023, and U.S. Provisional Patent Application No. 63/519,945, filed on Aug. 16, 2023, the contents of which are incorporated herein by reference in their entirety.

TECHNICAL FIELD

[0002] The following generally relates to cryptographic communications and, more particularly, to cryptographic protocols to establish end-to-end (E2E) secrets.

BACKGROUND

[0003] Delivering encryption keys, including strong E2E encryption keys, to a large number of endpoints is challenging. The threat of quantum computers being able to break widely deployed classical asymmetric cryptography has also increasingly become a concern.

[0004] State-of-the-art solutions include the following approaches: pre-shared keys (PSKs), key distribution centers (KDCs), and public-key infrastructures (PKI). Lately, post-quantum cryptography (PQC) approaches have emerged, either standalone or in a hybrid fashion, i.e., combined with tried-and-tested classical asymmetric cryptography.

Pre-Shared Keys

[0005] Pre-sharing is likely the oldest approach to distributing encryption keys. Keys or key shares can be delivered by trusted couriers which, depending on the circumstances, may have to be subject to substantial physical security measures. In some critical infrastructures, such as finance or certain military applications, this approach is still currently being heavily relied upon. PSKs are found to not scale, though, as the number of keys grows quadratically with the number of endpoints. Trustworthiness, reliability, cost, and the speed of couriers are additional issues. This led to a quest for automation, with KDCs being the first major step.

Key Distribution Centers

[0006] The advent of KDCs significantly improved scalability. Now, endpoints essentially only need a single (set of) pre-shared secret(s) which are used for authentication and encryption when communicating with a trusted (and, hopefully, trustworthy) KDC that facilitates E2E key generation and distribution. KDCs can be federated, with different entities serving different sets of endpoints.

[0007] The majority of widely deployed KDCs rely exclusively on symmetric cryptography, which is robust (even if attackers are equipped with powerful quantum computers, as long as key sizes are sufficiently large a breach is unlikely) and imposes only minimal performance and capacity requirements on the endpoints. The main caveat is that a compromise of a KDC is expected to have catastrophic consequences. Namely, in such a compromise, attackers are able to decrypt present, future, and sometimes also past E2E communication between endpoints, and they can also impersonate endpoints.

[0008] A prominent example of a KDC is Kerberos. However, Kerberos does not offer forward secrecy; its designers focused on authentication as a primary use case and tried to avoid further complicating the protocol, which already has a lot of moving parts (and, hence, attack surface) and, in addition, imposes strict requirements on time synchronization of the parties involved.

[0009] The use of asymmetric cryptography in the context of a PKI was expected to solve such KDC-related problems.

Public-Key Infrastructure

[0010] Asymmetric cryptography addresses some of the open issues via key establishment mechanisms such as Diffie-Hellman as well as through key encapsulation mechanisms (KEMs) that do not depend upon (pre-) shared secrets. In addition, public-key cryptography promises authentication techniques that do not require an endpoint to reveal its private keys to anybody. In practice, though, one is faced with a new set of issues.

[0011] For instance, endpoints often place trust in certificate authorities (CAs) to validate digital identities, i.e., the association of a public key with a particular entity. If compromised, a malicious CA allows for man-in-the-middle (MIM) attacks whereby an attacker impersonates an endpoint.

[0012] Furthermore, experience shows that asymmetric cryptography has been broken time and again for decades. The reasons are manyfold and may include bad choices of algorithms, software vulnerabilities, use of weak keys, and side-channel attacks. Even RSA, which rests on comparatively simple algebraic structures rendering it the world's most well-understood public-key crypto-system, is not immune [1].

(Paranoid) Post-Quantum Cryptography

[0013] The availability of sufficiently powerful quantum computers is expected to lead to all classical asymmetric cryptography becoming insecure. Its replacement, post-quantum cryptography (PQC), is still subject to cryptanalysis. Some promising-looking algorithms have been found to be susceptible to being broken. And, more is yet to come, even if some new algorithms actually do pass the test of time. Their reliance on ever more complex algebraic structures provide a larger attack surface when it comes to implementation errors, side-channel attacks, and keys that looked strong for some years until they turned out to be weak. The fact that the cryptographic community now has to analyze a growing variety of algorithms while attackers have access to new tools such as artificial intelligence (AI) exacerbates the problem.

[0014] It may also be noted that asymmetric cryptography cannot be relied upon to establish E2E keys with stringent long-term security requirements, regardless of whether classical or post-quantum cryptography (or a hybrid combination thereof) is used.

[0015] Paranoid post-quantum crypto is sometimes portrayed as a solution. It refers to the concurrent use of multiple PQC algorithms. While this offers some temporary relief (one should assume that eventually all real-world PQC implementations will exhibit vulnerabilities), it comes at a

huge implementation cost in terms of computational requirements (code size, runtime) and capacity (size of keys and signatures).

SUMMARY

[0016] The following addresses secure and efficient establishment of quantum-resistant long-term security (LTS) E2E cryptographic secrets to endpoints that are not directly connected via a quantum key distribution network (i.e., QKD links, possibly connected via trusted nodes).

[0017] A multimodal key delivery network (KDN) protocol is provided, which utilizes different types of symmetric and asymmetric cryptographic algorithms. Rather than hardwiring specific codes or key sizes, the following utilizes a crypto-agile approach that selects the respective parameters from continuously evolving guidelines by well-respected certification schemes such as the National Institute for Standards and Technology's (NIST's) Special Publications SP-800 series and German Bundesamt für Sicherheït in der Informationsverarbeitung's (BSI's) Technische Richtlinien (TRs).

[0018] In this context, one should consider the difference between a multimodal KDN protocol implementation in an endpoint and the subsequent use of the E2E key, e.g., for encryption. The latter is relatively straightforward in that all relevant encryption standards and products have interfaces that allow for the injection of PSKs. While PSKs did not solve the given key distribution problem, they can be useful as a modular approach to using E2E keys once established. Encryption devices can directly consume keys established using the proposed technique; they are decoupled from the actual implementation and crypto-agile adaptations of the multimodal KDN protocol.

[0019] Moreover, performance and efficiency (e.g., battery life) considerations often lead to high-priority requirements. To account for that, the multimodal KDN protocol can be configured in such a way that it requires only a small number of protocol steps and key material can be pre-generated; an endpoint can connect to the KDN to obtain multiple sets of pre-key data for a given destination endpoint. This can potentially allow for short connection latencies and good burst-mode performance.

[0020] In an aspect, a method for cryptographic establishment of shared secrets is disclosed. The method includes obtaining a first input share based on a hybrid key establishment method, obtaining a second input share from a key distribution network, and deriving, from the first input share and the second input share, a shared secret for use in cryptographic communication between an initiator and a respondent.

[0021] In example embodiments, obtaining the first input share from the KDN includes requesting, by the initiator, a dataset for establishing communication with the respondent, and receiving a responding dataset from the KDN. The responding dataset includes (1) a base key (i.e., the first input) derived from a pre-existing key secret between the respondent and a hub of the KDN, (2) an update counter and identifiers for the initiator and the hub, and (3) a message authentication code (MAC) authenticating the update counter and the identifiers, wherein the MAC authenticates via a pre-existing authentication secret shared by the respondent and the hub. The pre-existing key secret can be rolled by the respondent, in response to receiving the responding dataset. The request for the dataset transmitted by the initiator can trigger rolling of the pre-existing key secret by the hub.

[0022] In example embodiments, the method includes establishing a secure link between the initiator and another hub of the KDN by deriving, by the initiator, an authentication secret based on a pre-existing link secret shared by the other hub and the initiator. The method includes transmitting, by the initiator, at least the identifier of the initiator authenticated using the derived authentication secret, the derived authentication secret being able to validate the authenticated identifier transmitted by the initiator. The method includes receiving and confirming subsequent data from the other hub, the subsequent data being validated with the derived authentication secret and deriving a link key to establish the secure link based on the pre-existing link secret for subsequent communication between the initiator and the other hub. The hub and the other hub can be the same hub of the KDN. The at least the identifier of the initiator can include a nonce, and the subsequent data further comprises a second nonce, and the link key is derived based on the nonce and the second nonce.

[0023] The method can include updating the pre-existing link secret in response to the deriving the link key. The updated link secret can be used to establish subsequent secure links.

[0024] In example embodiments, the established secure link is used for secure asynchronous communication, and the method further comprises employing secure asynchronous communication based on the derived link key and a message counter.

[0025] In example embodiments, the initiator and the respondent are both endpoints, and the method further includes transmitting, by the initiator, pre-key data received from a hub the respondent is associated with, the transmitted pre-key data being processed by the respondent to obtain the first input. The method includes performing the hybrid key establishment method to generate the second input, and validating the first input and the second input. The respondent can be configured to, in processing the pre-key data, to update a secret associated with an update counter for communications between the respondent and the KDN, and compute the first input based on the updated secret.

[0026] In example embodiments, the hybrid key establishment method is based on exchanging via both a classical approach and a post-quantum approach. The exchange can include at least two exchanges, and the classical approach and the post-quantum approach exchanges are independent of one another. The method can include combining secrets resulting from the classical approach and the post-quantum approach to form the resulting hybrid shared secret.

[0027] In example embodiments, validating the first and second inputs includes deriving a confirmation key from the first input and the second input, and confirming, by the initiator, the validity of a message authentication code received from the respondent, the received message authentication code being generated at least in part with the confirmation key. The method includes transmitting, by the initiator, a message comprising a message authentication code generated at least in part based on the confirmation key to enable the respondent to validate the initiator transmitted message authentication code. The message authentication code can be generated based on outputs of the hybrid key establishment method.

[0028] The method can include storing unused first inputs and deleting used first inputs.

[0029] In another aspect, a method for associating two entities is disclosed. The method is for deriving shared secrets for use in cryptographic communications between the two entities, and includes based on a common shared base secret, deriving, a key secret, a link secret, and an authentication secret. The link secret can be used to communicate between the entities, and the key secret can be used to introduce a third entity known by at least one of the two entities to the other of the two entities, the key secret for establishing a shared secret between the third entity and the other of the two entities.

[0030] In another aspect, a system for cryptographic communications is disclosed. The system includes at least two endpoints and a key distribution network comprising a plurality of key distribution hubs connected to one another, at least one of the endpoints comprising a process and memory. The memory stores computer executable instructions that when executed by the processor cause the endpoint to perform any one of the methods discussed above.

[0031] In another aspect, a non-transitory computer readable medium (CRM) is disclosed. The CRM includes computer executable instructions for cryptographic communications, the computer executable instructions when executed by a processor causing the processor to perform any of the above methods.

[0032] In another aspect, a data processing apparatus is disclosed. The data processing apparatus includes means for carrying out the any of the above methods.

BRIEF DESCRIPTION OF THE DRAWINGS

[0033] Embodiments will now be described with reference to the appended drawings wherein:

[0034] FIG. **1** is a schematic diagram of a key distribution network.

[0035] FIG. **2** is a flow chart illustrating operations performed in establishing a secure link with a key distribution hub.

[0036] FIG. **3** is a flow chart illustrating operation performed in establishing a session key.

[0037] FIG. **4** is a flow chart illustrating a method for cryptographic communication.

DETAILED DESCRIPTION

[0038] As used herein, the term "obtain" can denote a derivation process (e.g., using the discussed hybrid key establishment method), or one or a sequence of transmissions to receive that "obtained information," (e.g., receiving pre-key data from a key distribution hub).

[0039] The following describes a protocol that allows two entities to obtain a common secret which can e.g., be used as a PSK in existing communication equipment.

[0040] At present, this is typically done using key establishment mechanisms based on asymmetric cryptography; sometimes using a combination of classical and PQC. While asymmetric cryptography can be used to establish short-lived ephemeral keys, it is not well suited as the root of trust for encryption keys with LTS requirements. On the other hand, KDCs sole reliance on more-robust all-symmetric cryptography bears substantial risks related to forward secrecy (FS) and identity theft when compromised.

[0041] In contrast, this disclosure includes a secret derived from two input shares in the presently described scheme. The first share is exchanged using a quantum-safe hybrid key establishment scheme, i.e., a combination of a classical key establishment and a PQC key establishment. The second share is obtained from a KDN having key distribution hubs (KDHs) connected via point-to-point links that are encrypted with symmetric keys which have been securely delivered out of band, e.g., as shown in FIG. **1**. In order to use the KDN, an endpoint needs to associate with a KDH employing a (one time) enrollment procedure. It can be appreciated that the KDN illustrated in FIG. **1** can be applied to various scenarios, for example, without limitation, telecommunications providers with an existing core network and separate access or peripheral network, organizations that operate a core network with high security requirements between major sites and a secondary network between minor sites (e.g., financial institutions, government installations), etc.

[0042] After enrollment, an endpoint can connect to the KDN to obtain (sets of) pre-key data for a given destination endpoint. This pre-key data can be cached and subsequently used to establish a common secret between the endpoints without the need for either of them to be connected to the KDN at that point. The pre-key data contains strong secrets that allow the endpoints to establish a secure long-term key without being part of the KDN itself. This makes it particularly suitable for mobile endpoints or 'last mile' connections.

[0043] The security of the presently described method rests on multiple modes of cryptographic operations that are strongly interwoven and may hereinafter be referred to as "multimodal cryptography".

[0044] Security properties of the proposed protocol include, without limitation:

[0045] E2E keys of strong computational security, even if all asymmetric crypto is broken.

[0046] Strong computational FS: even if all asymmetric crypto is broken, all endpoints and all KDHs are breached, past sessions are still secure.

[0047] Computational-strength post-compromise security (PCS).

[0048] Breaching any number of KDHs does not lead to endpoint identity theft.

[0049] Here, strong computational security refers to symmetric cryptography whereas the weaker computational security hinges on hybrid asymmetric cryptography, i.e., a combination of classical asymmetric cryptography and PQC.

[0050] KDHs can optionally be connected via QKD links. This allows a system to further strengthen the authenticated encryption used to protect key shares in transit.

Protocol Description

Notation

[0051] One can denote public keys by K, and private keys by k. For symmetric keys one can use K (for encryption and authentication keys), or $\mathcal{S}$ or $\mathcal{S}$ (for key derivation keys).

[0052] One can write $x \leftarrow E$ to denote assigning the result of the expression E to the variable x. One can write $x_0, \ldots, x_n \xleftarrow{\$} X$ to denote choosing values independently at random according to the probability distribution X and assigning

them to the variables $x_0, \ldots, x_n$. If X is a finite set, this notation can be abused to mean choosing random values uniformly from X.

[0053] One can use x ||y to denote the concatenation of two bitstrings x and y.

[0054] One can also denote a security parameter as n_{sec}. This determines the overall security strength (in bits) of the resulting end-to-end keys.

[0055] 1.0 Entities

[0056] 1.1 Key Distribution Hub

[0057] The KDHs make up the KDN. New KDHs are added to the KDN using pre-shared secrets delivered out of band. Within the KDN, KDHs can be directly connected to one another, i.e., they hold mutually shared secrets to secure communication with one another. Endpoints can enroll with the KDN, establishing an association, i.e., shared secrets between the endpoint and a KDH. KDHs broadcast the lists of associated endpoints—along with their public keys—to the KDN.

[0058] Secrets stored by KDH X can be summarized as follows:

[0059] the private key $k_{Auth,C}$ (X) for classical authenticated key establishment;

[0060] the private key $k_{Auth,PQ}$(X) for post-quantum authenticated key establishment;

[0061] for each directly connected KDH Y, a shared secret $\mathcal{S}_{Link}$(X, Y) to enable secure communication; and

[0062] for each associated endpoint P:

[0063] a shared secret $\mathcal{S}_{Link}$(X, P) to enable secure communication;

[0064] a shared secret $\mathcal{S}_{Key}$(X, P) used in key establishment with other endpoints; and

[0065] a shared secret key $K_{StaticAuth}$(X, P) to authenticate messages between X and P.

[0066] The public data stored by KDH X can be summarized as follows:

[0067] a unique identifier ID_X;

[0068] the public key $K_{Auth,C}$(X) for classical authenticated key establishment;

[0069] the public key $K_{Auth,PQ}$(X) for post-quantum authenticated key establishment;

[0070] for each directly connected KDH Y:

[0071] a unique identifier ID_Y;

[0072] the KDH's public key $K_{Auth,C}$(Y) for classical authenticated key establishment;

[0073] the KDH's public key $K_{Auth,PQ}$(Y) for post-quantum authenticated key

[0074] establishment; and

[0075] for each associated endpoint P:

[0076] a unique identifier ID_P;

[0077] an update counter $n_{KeySecret}$(X, P) for $\mathcal{S}_{Key}$(X, P);

[0078] the endpoint's public key $K_{Auth,C}$(P) for classical authenticated key establishment;

[0079] the endpoint's public key $K_{Auth,PQ}$(P) for post-quantum authenticated key establishment;

[0080] 1.2 Endpoint

[0081] The endpoints are entities which want to establish secure long-term keys with one another using the KDN. Each endpoint needs to associate with at least one KDH initially.

[0082] Secrets stored by endpoint P can be summarized as follows:

[0083] the private key $k_{Auth,C}$(P) for classical authenticated key establishment;

[0084] the private key $k_{Auth,PQ}$(P) for post-quantum authenticated key establishment;

[0085] for each KDH X with which P is associated:

[0086] a shared secret $\mathcal{S}_{Link}$(P, X) to enable secure communication;

[0087] a shared secret $\mathcal{S}_{Key}$(P, X) used in key establishment with other endpoints;

[0088] a set of base keys and their indices,

$$D_x=\{(i_1, S_{Key,i_1}, (P,X)), (i_2, S_{Key,i_2}(P,X)), \ldots\},$$

[0089] where the base keys $S_{Key,i}$(P, X) are derived from previous instances of $\mathcal{S}_{Key}$(P, X); and

[0090] a shared secret key $K_{StaticAuth}$(P, X) to authenticate messages between P and X.

[0091] Public data stored by endpoint P can be summarized as follows:

[0092] a unique identifier ID_P;

[0093] the public key $K_{Auth,C}$(P) for classical authenticated key establishment;

[0094] the public key $K_{Auth,PQ}$(P) for post-quantum authenticated key establishment;

[0095] for each KDH X with which P is associated:

[0096] the update counter $n_{KeySecret}$(P, X) for $\mathcal{S}_{Key}$(P, X), which can be transmitted in the clear between an initiating and target endpoint;

[0097] the KDH's public key $K_{Auth,C}$(X) for classical authenticated key

[0098] establishment; and

[0099] 2the KDH's public key $K_{Auth,PQ}$(X) for post-quantum authenticated key establishment.

2.0 Primitives

2.1 Hybrid Authenticated Key Exchanges

[0100] To perform a two-move authenticated key exchange, let an initiator A call

$$(X,x)\leftarrow\text{HybridKexInit}(k_{Auth,C}(A), k_{Auth,PQ}(A), K_{Auth,C}(B), K_{Auth,PQ}(B))$$

[0101] After receiving X from A, the responder B calls

$$(X', S)\leftarrow\text{HybridKexResp}(k_{Auth,C}(B), K_{Auth,PQ}(B), K_{Auth,C}(A), K_{Auth,PQ}(A), X)$$

[0102] Finally, having received X', the initiator A calls

$$S\leftarrow\text{HybridKexFinalize}(k_{Auth,C}(A), k_{Auth,PQ}(A), K_{Auth,C}(B), K_{Auth,PQ}(B), x, X')$$

[0103] Both parties end up with a shared secret S.

[0104] These three hybrid primitives are built from a two-move classical key exchange (InitC, RespC, FinalC) and a two-move post-quantum key exchange (InitPQ, RespPQ, FinalPQ).

[0105] The initiator A uses Init* to generate a secret x* and a public value X*, the responder B uses Resp* with X* to obtain the shared secret S* and a public value X'*, then the initiator A uses Final* with X' and x* to obtain S* as well. Both Resp* and Final* can also return fail in case of authentication failures. In this case, the parties abort the hybrid key exchange.

[0106] The parties perform the classical and post-quantum exchanges independent from one another, combining messages to keep the number of roundtrips low. They concatenate the resulting shared secrets to form the hybrid shared secret.

[0107] Below is a more detailed discussion of an example exchange of input shares to arrive at a shared secret S:

HybridKexInit($k_{Auth,\ C}$ (A), $k_{Auth,\ PQ}$ (A), $K_{Auth,\ C}$ (B), $K_{Auth,\ PQ}$ (B)):
1. Compute
(X_C, x_C) ← InitC ($k_{Auth,\ C}$ (A), $K_{Auth,\ C}$ (B))
2. and
(X_{PQ}, x_{PQ}) ← InitPQ ($k_{Auth,\ PQ}$ (A), $K_{Auth,\ PQ}$ (B)).
3. Return (X = (X_C, X_{PQ}), x = (x_C, x_{PQ})).
HybridKexResp($k_{Auth,\ C}$ (B), $k_{Auth,\ PQ}$ (B), $K_{Auth,\ C}$ (A), $K_{Auth,\ PQ}$ (A), X):
4. Parse (X_C, X_{PQ}) = X.
5. Compute
(X'_C, S_C) ← RespC($k_{Auth,\ C}$ (B), $K_{Auth,\ C}$ (A), X_C)
6. and
(X'_{PQ}, S_{PQ}) ← RespPQ($k_{Auth,\ PQ}$ (B), $K_{Auth,\ PQ}$ (A), X_{PQ}).
7. Return (X' = (X'_C, X'_{PQ}), S = S_C||S_{PQ}).
HybridKexFinalize($k_{Auth,\ C}$ (A), $k_{Auth,\ PQ}$ (A), $K_{Auth,\ C}$ (B), $K_{Auth,\ PQ}$ (B), x, X'):
8. Parse (x_C, x_{PQ}) = x and (X'_C, X'_{PQ}) = X'.
9. Compute
S_C ← FinalC($k_{Auth,\ C}$ (A), $K_{Auth,\ C}$ (B), x_C, X'_C)
10. and
S_{PQ} ← FinalPQ($k_{Auth,\ PQ}$ (A), $K_{Auth,\ PQ}$ (B), x_{PQ}, X'_{PQ}).
11. Return S = S_C||S_{PQ}.

[0108] There are several options for concrete instantiations of the classical and the post-quantum key exchanges. For example, for the classical key exchange, one may use any scheme which is eCK-secure [2] in the Random Oracle Model (ROM).

[0109] For the post-quantum key exchanges, to provide some examples, one may use any scheme which is eCK-secure against quantum adversaries in the Quantum Random Oracle Model (QROM), or FO_{AKE} [3] applied to a quantum-resistant public key encryption scheme.

2.2 Key Derivation

[0110] A system incorporating the present protocol can use key derivation functions (KDFs) to create the initial shared secrets in EnrollEndpoint (see section 3.1 below), update the shared secrets and derive other keys from the shared secrets. In each case, one can use high-entropy uniformly distributed base key material. Thus, to provide an example, one may use a simple pseudorandom function family (PRF)-based KDF (such as the ones in [4]) rather than an extract-then-expand construction like HKDF **8** 5,6**9** .

[0111] The primitive KDF (k,c) calls an example PRF-based KDF, using k as the base key material, c as the FixedInfo string, and an output size of n_{Sec}. The particular instances of c used hereafter are for illustrative purposes only and may be exchanged without limitation.

[0112] To update shared secrets, the protocol can call the KDF:

[0113] UpdateSecret(s)=KDF (s, "UpdateSecret")

2.3 Message Authentication and Confidentiality

[0114] Subsequent massages between the initiator and recipient can be authenticated with an Authenticated Encryption scheme with Associated Data (AEAD) with security properties described by Rogaway [7]: its ciphertexts can be indistinguishable from random bits under chosen-plaintext attacks, and the integrity tags shall be unforgeable.

[0115] One can denote by AEAD (Kκ, a, x) the primitive which uses this AEAD scheme to encrypt data x using the key κ, with associated data a.

[0116] The disclosed protocol, as an example, can fix a Message Authentication Code (MAC) scheme which is strongly existentially unforgeable under chosen-message attacks.

[0117] One can denote by MAC (κ, x) the primitive which computes this MAC with key κ over data x.

[0118] Potential choices include, without limitation: HMAC [8,9] for the MAC, and AES-GCM [10-12] for the AEAD scheme.

3.0 Procedures

3.1 EnrollEndpoint

[0119] Endpoint P enrolls with the KDN by establishing an association with KDH X. This procedure consists of two steps: secure establishment of a temporary base secret S and the derivation of long-lived shared secrets between P and X from the base secret S. Several options are feasible to establish such a base secret securely. Great care has to be taken to ensure the base secret S is established in a way that meets or exceeds the security requirement of the consumer of the E2E keys established with the multimodal protocol. For example, P and X can use a hybrid key establishment method to establish the base secret S, which can include using a secure link (e.g., at least one of a temporary, and physical link) in the first step.

P computes
(E_P, e_P) ← HybridKexInit($k_{Auth,\ C}$ (P), $k_{Auth,\ PQ}$ (P), $K_{Auth,\ C}$ (X), $K_{Auth,\ PQ}$ (X));
and sends ($K_{Auth,\ C}$ (P), $K_{Auth,\ PQ}$ (P), E_P) to X.
X computes:
(E_X, S) ← HybridKexResp($k_{Auth,\ C}$ (X), $k_{Auth,\ PQ}$ (X), $K_{Auth,\ C}$ (P), $K_{Auth,\ PQ}$ (P), E_P)
and sends ($K_{Auth,\ C}$ (X), $K_{Auth,\ PQ}$ (X), E_X) to P;
P computes
S ← HybridKexFinalize($k_{Auth,\ C}$ (P), $k_{Auth,\ PQ}$ (P), $K_{Auth,\ C}$ (X), $K_{Auth,\ PQ}$ (X), e_P, E_X);

[0120] In the second step, P and X derive the long-lived shared secrets in the following way:

$\mathcal{S}_{Link}(P, X)=\mathcal{S}_{Link}(X, P)\leftarrow KDF(S, \text{“LinkSecret”})$,

$\mathcal{S}_{Key}(P, X)=\mathcal{S}_{Key}(X, P)\leftarrow KDF(S, \text{“KeySecret”})$,

$K_{StaticAuth}(P, X)=K_{StaticAuth}(X, P)\leftarrow KDF(S, \text{“StaticAuth”})$,

[0121] and set $n_{KeySecret}(P, X)=n_{KeySecret}(X, P)\leftarrow 0$. Finally, P sets $D_X\leftarrow\emptyset$; and

[0122] P and X perform EstablishLink (see section 3.2 below) to confirm the shared secrets just established.

3.2 EstablishLink

[0123] FIG. **2** shows an example method of establishing a link between an initiator and a key distribution hub. More specifically, an initiator A establishes a secure link with KDH X. The initiator A can be either an endpoint P or another KDH. A shares a link secret $\mathcal{S}_{Link}(A, X)$ with X. In the case of an endpoint, this means A previously has associated with X using EnrollEndpoint (see section 3.1 above). In the case of another KDH, shared secrets have been established between them. The following operations are shown in FIG. **2**.

[0124] 1. A derives the MAC key:

$\kappa_M\leftarrow KDF(\mathcal{S}_{Link}(A, X), \text{“MAC”})$

[0125] 2. A samples a nonce $N_A\xleftarrow{\$}\{0,1\}^{n_{Sec}}$ and computes the MAC:

$M_A\leftarrow MAC(\kappa_M, ID_A||N_A)$

[0126] 3. A sends (ID_A, N_A, M_A) to X.

[0127] 4. X derives the MAC key and verifies the MAC M_A. If this fails, it computes a tentatively updated link secret:

$S\leftarrow UpdateSecret(\mathcal{S}_{Link}(X, A))$

[0128] and recomputes the MAC key using this as the base key. If X can verify M_A using the recomputed MAC key, it updates its link secret:

$\mathcal{S}_{Link}(X, A)\leftarrow S$

[0129] to resync with A. Otherwise, it aborts the procedure.

[0130] 5. X initializes the link's sequence counter $s\leftarrow 0$, samples a nonce $N_X\xleftarrow{\$}\{0,1\}^{n_{Sec}}$ and computes the MAC, link key, and ciphertext:

$M_X\leftarrow MAC(\kappa_M, N_X)$

$\kappa_{Link}KDF(\mathcal{S}_{Link}(X, A), N_A||N_X)$, and

$C_X\leftarrow AEAD(\kappa_{Link}, s, x)$, where x is some payload (e.g., a link ID for later reference)

[0131] 6. X sends (N_X, M_XC_X) to A.

[0132] 7. A verifies the MAC M_X, derives the link key and uses it to verify and decrypt C_X. If either verification fails, A aborts the procedure.

[0133] 8. A increments the link's sequence counter $S\leftarrow S+1=1$, computes the ciphertext:

$C_A\leftarrow AEAD(\kappa_{Link}, s, a)$

[0134] of some payload a and sends it to X.

[0135] 9. X verifies and decrypts the message. If this fails, it aborts the procedure.

[0136] 10. A and X update their link secrets:

$\mathcal{S}_{Link}(A, X)\leftarrow UpdateSecret(\mathcal{S}_{Link}(A,X))$, $\mathcal{S}_{Link}(X, A)\leftarrow UpdateSecret(\mathcal{S}_{Link}(X,A))$

[0137] Now both parties use AEAD $(\kappa_{Link}, s, \cdot)$ to secure further communication, where s is the sequence counter. initially set to s=0 in step **5**, it is incremented for every message. Whenever A sends a request with sequence counter s, it checks that the response's sequence counter is s+1. Otherwise, the link is closed and re-established. Similarly, the link is closed if a decryption attempt fails due to an invalid integrity tag.

[0138] Note that a link may be used for asynchronous communication, and responses to multiple requests do not have to arrive in order: A is free to send messages with sequence counters s, s+2, s+4 without waiting for responses in between. When the responses come back, A matches the responses to the requests using either the sequence counters or other means provided by the transport layer. The parties need to close the link if at any point in time a sequence counter is reused, or A receives a response with sequence counter s+1 and it did not send a request with counter s, or if a request and response can be matched by means other than the sequence counter and the response's counter is not the increment of the request's counter.

[0139] To avoid race conditions on the sequence counter, A only sends requests, and X sends a single response for each request. If X needs to send a request to A, the parties establish a link with reversed roles for that purpose.

[0140] One can serialize sequence counters as 32-bit unsigned integers in little-endian byte order. A link's sequence counter must always be $s<2^{32}$, otherwise the link must be closed and re-established. (In fact, links should usually be closed much sooner than that.)

[0141] Two parties' link secrets can get out of sync if the message sent in step **8** is lost. In this case, only A updates its secret. On the next EstablishLink call, X will resync in step **4**.

[0142] If both parties are KDHs, they may also switch roles after getting out of sync. To allow resyncing in this situation, an initiating KDH needs to retry EstablishLink calls with a tentatively updated link secret whenever the responding KDH aborts the procedure at step **4**, as this may indicate that the KDHs got out of sync before switching roles. If the initiating KDH then receives the message sent in step **6**, the retry succeeded, and it must overwrite its link secret by the tentatively updated copy to complete the resync (the secret will proceed to be updated again in step **10**).

3.3 GetPreKeyData

[0143] An endpoint P wants to establish a shared secret with a target endpoint Q. This happens in two phases. First, P requests a PreKeyData object for a connection with Q from the KDN:

[0144] P opens a secure connection to a KDH X with which it is associated using EstablishLink (see section 3.2) and sends ID_Q to X.

[0145] If Q is not associated with KDH X, X identifies a KDH Y, with which Q is associated and opens a secure connection to Y using EstablishLink (see section 3.2). (If such secure connections already exist, they may be reused.) X sends ID_P and ID_Q to Y. Otherwise, i.e. P and Q are

associated with the same KDH, in the following both X and Y refer to that KDH, which performs all subsequent steps itself.

[0146] Y derives a base key and computes a MAC over its ID and its key secret's update counter:

$$S_Y \leftarrow KDF(\mathcal{S}_{Key}(Y, Q), \text{“SessionKeyBase”})$$

$$M \leftarrow MAC(\kappa_{StaticAuth}(Y, Q), ID_P||D_Y||n_{KeySecret}(Y, Q))$$

[0147] Using the secure connections, Y sends

$$PreKeyData_Y=(ID_P, ID_Y, n_{KeySecret}(Y, Q), M)$$

[0148] and the base key S_Y back to P. If any connection is closed due to a sequence counter mismatch in the course of delivery of this response, abort the procedure. If the first ID in the returned PreKeyData is not ID_P, P also aborts the procedure.

[0149] Y rolls the secret:

$$\mathcal{S}_{Key}(Y, Q \leftarrow UpdateSecret(\mathcal{S}_{Key}(Y, Q))$$

$$n_{KeySecret}(Y, Q) \leftarrow n_{KeySecret}(Y, Q)+1$$

3.4 EstablishSessionKey

[0150] Once an endpoint P has obtained PreKeyData, and the base key S_Y for a connection with target endpoint Q (via GetPreKeyData (see section 3.3 above)), it can establish a shared session key with Q.

[0151] P computes

$$(E_{PQ}, e_P) \leftarrow HybridKexInit(k_{Auth,C}(P), k_{Auth,PQ}(P), K_{Auth,C}(Q), K_{Auth,PQ}(Q))$$

[0152] and sends (E_{PQ}, $PreKeyData_Y$) to Q.

[0153] Q verifies the MAC M_Y contained in $PreKeyData_Y$. If this fails, it aborts the procedure.

[0154] Denote the key secret update counter contained in $PreKeyData_Y$ by n, and Q's own counter $n_{KeySecret}(Q, Y)$ by q. If q≤n, then Q computes:

$$S_{Key,i}(Q, Y) \leftarrow KDF(\mathcal{S}_{Key}(Q, Y), \text{“SessionKeyBase”})$$

$$D_Y \leftarrow D_Y \cup \{(i, S_{Key,i}(Q, Y))\} \text{ for each } i \in \{q, \ldots, n\}$$

$$\mathcal{S}_{Key}(Q, Y) \leftarrow UpdateSecret(\mathcal{S}_{Key}(Q,Y))$$

[0155] and sets $n_{KeySecret}(Q, Y) \leftarrow n+1$.

[0156] If the tuple $t_n=(n, S_{Key,n}(Q, Y))$ is not in D_Y, the base key corresponding to the received counter has already been used and Q aborts the procedure. Otherwise, it stores the base key temporarily as $S_{SKey,n}(Q, Y)$ and removes the pair t_n from D_Y.

[0157] Q computes the responder's part of the key exchange, derives a confirmation key and computes a MAC: $K_{Auth,PQ}(Q)$

$$(E_{QP}, S_{PQ}) \leftarrow HybridKexResp(k_{Auth,C}(Q), k_{Auth,PQ}(Q), K_{Auth,C}(P), K_{Auth,PQ}(P), E_{PQ})$$

$$\kappa_{Conf} \leftarrow KDF(S_Y||S_{PQ}, \text{“ConfirmationKey”})$$

$$M_Q \leftarrow MAC(\kappa_{Conf}, \text{“ConfirmResp”}||E_{PQ}||E_{QP})$$

[0158] and sends (E_{QP}, M_Q) to P.

[0159] P computes

$$S_{PQ} \leftarrow HybridKexFinalize(k_{Auth,C}(P), k_{Auth,PQ}(P), K_{Auth,C}(Q), K_{Auth,PQ}(Q), e_P p, E_{QP})$$

[0160] and derives κ_{Conf}. Using this, it verifies M_Q. If the verification fails, P aborts the procedure.

[0161] P computes

$$M_P \leftarrow MAC(\kappa_{Conf}, \text{“ConfirmInit”}||E_{PQ}||E_{QP})$$

[0162] and sends it to Q.

[0163] Q verifies M_P. If this fails, it aborts the procedure.

[0164] Both P and Q can now derive the session key:

$$\kappa_{Session} \leftarrow KDF(S_Y||S_{PQ}, \text{“SessionKey”})$$

[0165] Storing unused base keys derived from the shared key secret allows endpoints to handle out-of-order requests while rolling the shared key secret early. In case an endpoint's secrets are leaked at any point in time, the previous instances of the shared key secret as well as the used base keys have already been deleted, maintaining forward secrecy for completed key exchanges.

[0166] Symmetric-key re-keying in general has long been an instrument used by cryptographers [13]. Its application towards out-of-order message handling in particular has been adapted from Signal's Double Ratchet algorithm [14].

[0167] The GetPreKeyData and EstablishSessionKey protocols as discussed above are illustrated in FIG. **3**.

[0168] Referring now to FIG. **4**, a flow diagram of a method for establishment of cryptographic secrets is shown. At block **402**, a first input share based on a hybrid key encapsulation method is obtained. At block **404**, a second input share is obtained from a key distribution network. At block **406**, a shared secret for use in cryptographic communication is derived based on the first input share and the second input share.

[0169] For simplicity and clarity of illustration, where considered appropriate, reference numerals may be repeated among the figures to indicate corresponding or analogous elements. In addition, numerous specific details are set forth in order to provide a thorough understanding of the examples described herein. However, it will be understood by those of ordinary skill in the art that the examples described herein may be practiced without these specific details. In other instances, well-known methods, procedures and components have not been described in detail so as not to obscure the examples described herein. Also, the description is not to be considered as limiting the scope of the examples described herein.

[0170] It will be appreciated that the examples and corresponding diagrams used herein are for illustrative purposes only. Different configurations and terminology can be used without departing from the principles expressed herein. For instance, components and modules can be added, deleted, modified, or arranged with differing connections without departing from these principles.

[0171] It will also be appreciated that any module or component exemplified herein that executes instructions may include or otherwise have access to computer readable media such as transitory or non-transitory storage media, computer storage media, or data storage devices (removable and/or non-removable) such as, for example, magnetic disks, optical disks, or tape. Computer storage media may include volatile and non-volatile, removable and non-removable media implemented in any method or technology for storage of information, such as computer readable instructions, data structures, program modules, or other data. Examples of computer storage media include RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other non-transitory computer readable medium which can be used to store the

desired information and which can be accessed by an application, module, or both. Any such computer storage media may be part of the KDHs, KDN, other device in the system, any component of or related thereto, etc., or accessible or connectable thereto. Any application or module herein described may be implemented using computer readable/executable instructions that may be stored or otherwise held by such computer readable media.

[0172] The steps or operations in the flow charts and diagrams described herein are provided by way of example. There may be many variations to these steps or operations without departing from the principles discussed above. For instance, the steps may be performed in a differing order or in parallel, or steps may be added, deleted, or modified.

[0173] Although the above principles have been described with reference to certain specific examples, various modifications thereof will be apparent to those skilled in the art as having regard to the appended claims in view of the specification as a whole.

REFERENCES

[0174] [1] D. Boneh, "Twenty years of attacks on the rsa cryptosystem," Notices of the American Mathematical Society, vol. 46, pp. 203-212, 1999.

[0175] [2] B. LaMacchia, K. Lauter, and A. Mityagin, "Stronger security of authenticated key exchange," in Provable Security (W. Susilo, J. K. Liu, and Y. Mu, eds.), (Berlin, Heidelberg), pp. 1-16, Springer Berlin Heidelberg, 2007.

[0176] [3] K. Hovelmanns, E. Kiltz, S. Schage, and D. Unruh, "Generic authenticated key exchange in the quantum random oracle model." Cryptology ePrint Archive, Paper 2018/928, 2018. https://eprint.iacr.org/2018/928.

[0177] [4] L. Chen, "Recommendation for key derivation using pseudorandom functions," Aug2022.

[0178] [5] H. Krawczyk, "Cryptographic extraction and key derivation: The hkdf scheme," Advances in Cryptology—CRYPTO 2010, p. 631-648, 2010.

[0179] [6] E. Barker, L. Chen, and R. Davis, "Recommendation for key-derivation methods in keyestablishment schemes," Aug 2020.

[0180] [7] P. Rogaway, "Authenticated-encryption with associated-data," Proceedings of the 9th ACM conference on Computer and communications security, 2002.

[0181] [8] H. Krawczyk, M. Bellare, and R. Canetti, "Rfc2104: Hmac: Keyed-hashing for message authentication," 1997.

[0182] [9] The keyed-hash message authentication code (hmac)," Tech. Rep. Federal Information Processing Standards Publications (FIPS PUBS) 198-1, U.S. Department of Commerce, Washington, D.C., July 2008.

[0183] [10] D. A. McGrew and J. Viega, "The galois/counter mode of operation (gcm)," Jan 2004.

[0184] [11] D. A. McGrew and J. Viega, "The security and performance of the galois/counter mode (gcm) of operation," Progress in Cryptology—INDOCRYPT 2004, p. 343-355, 2004.

[0185] [12] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac," Nov 2007.

[0186] [13] M. Abdalla and M. Bellare, "Increasing the lifetime of a key: A comparative analysis of the security of re-keying techniques," Advances in Cryptology—ASIACRYPT 2000, p. 546-559, 2000.

[0187] [14] T. Perrin and M. Marlinspike, "The double ratchet algorithm," Nov 2016.

1. A method for establishment of cryptographic secrets, the method comprising:

obtaining a first input share from a key distribution network (KDN); and

obtaining a second input share based on a hybrid key establishment method;

deriving, from the first input share and the second input share, a shared secret for use in cryptographic communication between an initiator and a respondent.

2. The method of claim **1**, wherein obtaining the first input share from the KDN comprises:

requesting, by the initiator, a dataset for establishing communication with the respondent; and

receiving a responding dataset from the KDN comprising (1) a base key derived from a pre-existing key secret between the respondent and a hub of the KDN, (2) an update counter and identifiers for the initiator and the hub, and (3) a message authentication code (MAC) authenticating the update counter and the identifiers,

wherein the MAC authenticates via a pre-existing authentication secret shared by the respondent and the hub.

3. The method of claim **2**, further comprising rolling the pre-existing key secret, by the respondent, in response to receiving the responding dataset.

4. The method of claim **2**, wherein the initiator transmitting the request for the dataset triggers rolling of the pre-existing key secret by the hub.

5. The method of claim **2**, further comprising establishing a secure link between the initiator and another hub of the KDN by:

deriving, by the initiator, an authentication secret based on a pre-existing link secret shared by the other hub and the initiator;

transmitting, by the initiator, at least the identifier of the initiator authenticated using the derived authentication secret, the derived authentication secret being able to validate the authenticated identifier transmitted by the initiator;

receiving and confirming subsequent data from the other hub, the subsequent data being validated with the derived authentication secret

deriving a link key to establish the secure link based on the pre-existing link secret for subsequent communication between the initiator and the other hub.

6. The method of claim **5**, wherein the hub and the other hub are the same hub of the KDN.

7. The method of claim **5**, wherein at least the identifier of the initiator further comprises a nonce, and the subsequent data further comprises a second nonce, and the link key is derived based on the nonce and the second nonce.

8. The method of claim **7**, further comprising updating the pre-existing link secret in response to the deriving the link key.

9. The method of claim **8**, further comprising using the updated link secret to establish subsequent secure links.

10. The method of claim **5**, wherein the established secure link is used for secure asynchronous communication, and the method further comprises employing secure asynchronous communication based on the derived link key and a message counter.

11. The method of claim 1, wherein the initiator and the respondent are both endpoints, the method further comprising:

transmitting, by the initiator, pre-key data received from a hub the respondent is associated with, the transmitted pre-key data being processed by the respondent to obtain the first input;

performing the hybrid key establishment method to generate the second input; and

validating the first input and the second input.

12. The method of claim 11, wherein the respondent is configured to, in processing the pre-key data, to:

update a secret associated with an update counter for communications between the respondent and the KDN; and

compute the first input based on the updated secret.

13. The method of claim 1, wherein the hybrid key establishment method is based on exchanging via both a classical approach and a post-quantum approach.

14. The method of claim 13, wherein the exchange comprises at least two exchanges, and the classical approach and the post-quantum approach exchanges are independent of one another.

15. The method of claim 13, comprising combining secrets resulting from the classical approach and the post-quantum approach to form the resulting hybrid shared secret.

16. The method of claim 11, wherein validating the first and second inputs comprises:

deriving a confirmation key from the first input and the second input; and

confirming, by the initiator, the validity of a message authentication code received from the respondent, the received message authentication code being generated at least in part with the confirmation key;

transmitting, by the initiator, a message comprising a message authentication code generated at least in part based on the confirmation key to enable the respondent to validate the initiator transmitted message authentication code.

17. The method of claim 16, wherein the message authentication code is generated based on outputs of the hybrid key establishment method.

18. The method of claim 12, further comprising storing unused first inputs and deleting used first inputs.

19. A method for associating two entities comprising deriving shared secrets for use in cryptographic communications between the two entities, the method comprising:

based on a common shared base secret, deriving:

a key secret,

a link secret,

an authentication secret.

20. The method of claim 19, wherein the link secret is used to communicate between the entities, and the key secret is used to introduce a third entity known by at least one of the two entities to the other of the two entities, for establishing a shared secret between the third entity and the other of the two entities.

21. A system for cryptographic communications, the system comprising at least two endpoints and a key distribution network comprising a plurality of key distribution hubs connected to one another, at least one of the endpoints comprising a process and memory, the memory storing computer executable instructions that when executed by the processor cause the endpoint to perform operations comprising:

obtaining a first input share from the key distribution network;

obtaining a second input share based on a hybrid key establishment method; and

deriving, from the first input share and the second input share, a shared secret for use in cryptographic communication between an initiator and a respondent.

22. A non-transitory computer readable medium comprising computer executable instructions for cryptographic communications, the computer executable instructions when executed by a processor causing the processor to perform operations comprising:

obtaining a first input share from a key distribution network (KDN); and

obtaining a second input share based on a hybrid key establishment method;

deriving, from the first input share and the second input share, a shared secret for use in cryptographic communication between an initiator and a respondent.

* * * * *